

Last modified on

Sports Data Soccer API - Match Events Possession Value Feed (MA32)

Get Possession Values mapped to Events happening in a particular match that's queried for.

Overview

The Soccer **Match Events Possession Value** feed provides all the traditional data such as match information, competition information, event information and team details clubbed with new possession value information.

For more details refer: <https://www.theanalyst.com/eu/2021/03/what-is-possession-value/>

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our **FAQs**. We also recommend that you take a look at our **API Overview and Authentication Guide**. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

POLLING FREQUENCY

Once per minute in case a match that's queried for is "live".

Match Events Possession Value (MA32) Guide

USAGE	FEED
GET Match Events Possession Value by fixture UUID.	/soccerdata/matcheventspossessionvalues/{outletAuthKey}/{fixtureUUID}?{queryParameters}
GET Match Events Possession Value by fixture.	/soccerdata/matcheventspossessionvalues/{outletAuthKey}?fx={fixtureUUID}&{queryParameters}
GET Match Events Possession Value by type	/soccerdata/matcheventspossessionvalues/{outletAuthKey}/{fixtureUUID}?{queryParameters}&type={type value};

If your request URL is as follows (does not contain a fixture UUID), the feed returns an error:
 /soccerdata/matcheventspossessionvalues/{outletAuthKey}

How can I get fixture and other entity UUIDs?

Use the **Tournament Calendar (OT2)** or **Fixtures and Results (MA1)** feeds to get fixture metadata, including its unique ID (UUID), linked contestants, competition, tournament calendar and other entities.

Query Parameters And Example Request Parameters

You must query the `/matcheventspossessionvalues` feed by a fixture (match) UUID. This feed also makes use of the **Global query parameters**, and you can combine them with the following query parameters (some of which can be used in conjunction with each other).

Legacy IDs: You can use the `fx` parameter to filter by legacy IDs - only one ID is supported (you cannot pass multiple IDs in a comma-separated list).

Making successful requests: In our guides, we use HTTPS syntax for our example `GET` requests (rather than `cURL`, for example). All URIs are case-sensitive - this means that URI strings must contain the correct case and operators. You **MUST** format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL

(`{protocol}://{requestDomain}/{feedResource}`), in the **main guide** and **API Overview and Authorisation guide**.

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` (Outlet Authentication Key), query parameters for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format). If required, it should also include an entity UUID (such as a fixture/match UUID).
- Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or - if a parameter supports it - `,` (multiple values, 'OR') or `!` ('NOT').
- Use the required `_` (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces `{ }` from your calls (we include these as an example only).
- If using the JSONP format (`_fmt=jsonp`) you **MUST** use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
<code>fixtureUuid</code>	GET Match Events Possession Value for a match by the specified fixture UUID (path parameter method). Pass the fixture UUID in the request URL directly as the <code>{assetId}</code> (asset/resource ID). <pre>GET https://api.performfeeds.com/soccerdata/matcheventspossessionvalues/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}</pre>
<code>fx</code>	Get Match Events Possession Value for a match by the specified fixture UUID (query parameter method). Pass the fixture UUID to the <code>fx</code> parameter.

Query parameter	Value and description
	GET https://api.performfeeds.com/soccerdata/matcheventspossessionvalues/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&fx={fixtureUuid}
type	Get Match Events Possession Value only of a specific event type(s) in the fixture, such as shots, goals, and crosses. Pass the event type ID(s) to the <code>type</code> parameter. You can specify multiple event <code>typeId</code> values to the <code>type</code> parameter in a comma-separated list (up to a maximum of 20 values). single event <code>typeid</code> value: GET https://api.performfeeds.com/soccerdata/matcheventspossessionvalues/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}&ctst={contestantUuid}&prsn={personUuid}&type={typeid} multiple event <code>typeid</code> values: GET https://api.performfeeds.com/soccerdata/matcheventspossessionvalues/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}&ctst={contestantUuid}&prsn={personUuid}&type={typeid},{typeid},{typeid}
_lcl=en-op	Add this parameter-value to return assets with the Opta namings, in English, in the response (instead of Opta Core namings). When the call does not include the <code>_lcl=en-op</code> parameter-value, the feeds will continue to work as they do currently; one example is that the response will be in the default language of English unless the call contains the <code>_lcl</code> parameter and a locale code value to request the response in a different locale/language. Note: You can use this parameter in combination with other valid parameters and values in the request. However, you cannot use it in combination with a locale/language code and <code>_lcl</code> parameter. GET https://api.performfeeds.com/soccerdata/matcheventspossessionvalues/{outletAuthKey}/fixtureuid?_rt={mode}&_fmt={dataFormat}&_lcl=en-op&{queryParams} Do not supply both the <code>_lcl</code> (with a locale/language code value) and <code>_lcl=en-op</code> in the same call - this is not supported; otherwise, the feed will return an error and code 10203 (Ambiguous/Invalid request parameters) in the response due to a duplicated <code>_lcl</code> parameter.

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- XML
- JSON

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

- The root element can be `matchEventsPossessionValues` - see the table below.

- In the scenario where the match has not yet been played (for example, it is a scheduled fixture to be played in the future), not all attributes listed here will be present and we may provide placeholder values for some attributes. For example, the scores, winner, contestant name, position, and ID may not be known (the contestant `id` attribute may also not be returned). However, the feed will be updated with this information once it is available.

▶<matchEventsPossessionValues> (XML only)

▶<matchInfo>

▶<description>

▶<sport>

▶<ruleset>

▶<competition>

▶<country>

▶<tournamentCalendar>

▶<stage>

▶<series>

▶<contestants> (XML only)

▶<contestant>

▶<country>

▶<venue>

▶<liveData>

▶<matchDetails>

▶<periods> (XML only)

▶<period>

▶<suspensions>

▶<suspension>

▶<scores>

▶<ht>

▶<ft>

▶<et>

▶<pen>

▶<total>

▶<totalUnconfirmed>

▶<events> (XML only)

▶<event>

▶<possessionValue>

Event Types

Events are delivered as an array (under `events` for XML or `events` for JSON). When an event occurs during a match, it is assigned a unique Event ID. For each unique Event ID, there is an associated Event Type ID (`typeId`) which classifies the general event type that has occurred.

Use this table to look-up an Event Type ID and its description. list.

Event Definitions

We have a long-established, consistent list of 'Event Definitions' in football that are adhered to across all data collection centres. You can find the most up-to-date definitions in the [Opta Glossary](#) , where we maintain an up-to-date list of the main football events logged by Opta.

▶ **SHOW/HIDE: event typeId | name | description**

Useful Queries

▶ **SHOW/HIDE: Type_id | outcome | Extra info | Excluding Events With These Qualifiers:**

Appendix

This section helps give further context to some Events, such as pitch coordinates and other data. Click on the buttons below to view each section.

Appendix 1 - Pitch Coordinates And Direction Of Play

▶ **SHOW/HIDE**

Appendix 2 - Goalmouth Coordinates

▶ **SHOW/HIDE**

Appendix 3 - Pitch Zones

▶ **SHOW/HIDE**

Appendix 4 - Playing Positions

► **SHOW/HIDE**

Appendix 5 - Formations Explained

► **SHOW/HIDE**

Appendix 6 - Kit IDs

► **SHOW/HIDE**

Appendix 7 - Coordinates Transformation

► **SHOW/HIDE**